

Building Intelligent, Tool-Using AI Systems

What Is an AI Agent?

An AI agent is a system that can:

- Perceive a task or environment
- Reason about what to do next
- Select an action or tool
- Observe the result
- Continue until the task is complete

Key difference from simply using a LLM is an agent does not only generate text; it uses that text to decide what action to take.

The Core Agent Loop

1. Receive a user request
2. Interpret the goal
3. Decide whether to answer directly or call a tool
4. Execute the tool call
5. Read the tool result
6. Continue reasoning
7. Stop when a final answer is ready

This is the pattern behind most modern agent systems.

Why Tool Use Matters

Without tools, an LLM is limited to the text already in its context window.

With tools, an agent can:

- Look up current information
- Query databases
- Call APIs
- Send notifications
- Run code
- Trigger workflows

Tools turn static language models into interactive software systems.

Tool Schemas in the ReAct Example

```
TOOL_SCHEMAS = [  
  {  
    "type": "function",  
    "function": {  
      "name": "get_weather",  
      "description": "Get current weather for a city using Open-Meteo.",  
      "parameters": {  
        "type": "object",  
        "properties": {  
          "location": {"type": "string"}  
        },  
        "required": ["location"]  
      }  
    }  
  }  
]
```

Concrete Tool Functions

```
def get_weather(location: str) -> dict:
    coords = geocode_city(location)
    forecast = fetch_open_meteo(coords)
    return {
        "location": location,
        "temp_f": forecast["temperature_2m"],
        "conditions": weather_code_to_label(forecast["weather_code"]),
        "wind_speed_mph": forecast["wind_speed_10m"]
    }

def search_database(query: str) -> list[dict]:
    q = query.lower()
    return [p for p in PRODUCTS if q in p["name"].lower() or q in p["category"].lower()]
```

Direct Tools vs Agent Logic

Separate two concerns:

- Tools: the functions that do useful work

- Agent: the logic that chooses when to call them

This separation makes the system easier to test.

You can test tools independently.

You can test the reasoning loop independently.

You can later swap direct local tools for remote tools or MCP tools.

ReAct: Reasoning + Acting

ReAct combines reasoning with tool use.

Pattern:

Thought -> choose an action

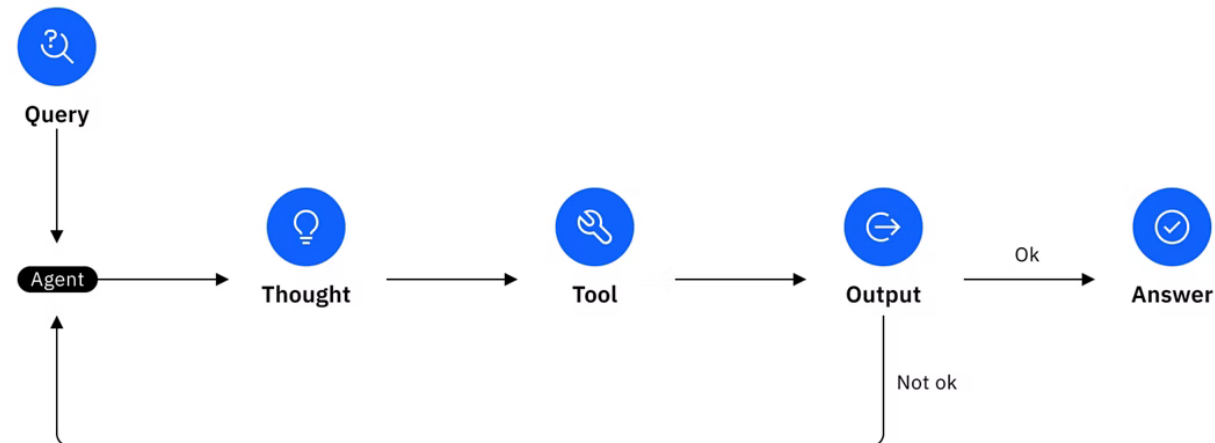
Action -> call a tool

Observation -> inspect the result

Thought -> decide what to do next

Repeat until done

ReAct is useful because it makes the structure of the workflow explicit.



ReAct Trace Example

```
$ python agent.py --task "What is the weather in San Francisco and should I bring a jacket?" --verbose
```

Tool: get_weather

Arguments: {"location": "San Francisco"}

Result: {"location": "San Francisco", "temp_f": 58.4, "conditions": "partly cloudy",
"wind_speed_mph": 7.1}

Final Answer: San Francisco is cool and partly cloudy, so bringing a light jacket is a good idea.



+



Representing Agent Steps

```
@dataclass
class ToolCall:
    tool_name: str
    arguments: dict[str, Any]
    result: Any
```

```
@dataclass
class AgentResult:
    final_answer: str
    tool_calls: list[ToolCall]
```

These records make it easy to inspect what happened during execution.

A Minimal ReAct Agent Class

```
class ReActAgent:
    def __init__(self, max_turns: int = 5):
        self.max_turns = max_turns

    def run(self, task: str) -> AgentResult:
        messages = [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": task},
        ]
        # call the LLM
        # execute any requested tools
        # append tool results
        # stop when the model returns a final answer
```

Calling OpenRouter

```
assistant_message = chat_completion(  
    messages=messages,  
    tools=TOOL_SCHEMAS,  
    tool_choice="auto",  
)  
  
if assistant_message.get("tool_calls"):  
    execute_requested_tools(...)  
else:  
    return assistant_message["content"]
```

Executing Requested Tools

```
for tool_call in raw_tool_calls:
    function_data = tool_call["function"]
    tool_name = function_data["name"]
    arguments = json.loads(function_data.get("arguments", "{}"))

    result = TOOLS[tool_name](**arguments)
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call["id"],
        "content": json.dumps(result)
    })
```

What an LLM Adds

A rule-based agent hard-codes the decision step.

An LLM-based agent generalizes it:

- Which tool should I use?

- What arguments should I send?

- Do I have enough information yet?

- Should I chain another tool call?

The loop stays the same; the decision policy becomes model-driven.

Designing Good Tools

Strong tools are:

- Single-purpose
- Named clearly
- Described precisely
- Typed with an explicit schema
- Safe to call repeatedly
- Consistent in their outputs

Poorly designed tools make even a strong model behave badly.

Good Tool Description Example

```
{
  "type": "function",
  "function": {
    "name": "search_database",
    "description": "Search the product catalog by keyword in product name or category and return matching records.",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {"type": "string", "description": "Keyword such as laptop or tablet"}
      },
      "required": ["query"]
    }
  }
}
```


Structured Outputs Help Agents

```
[  
  {"id": 1, "name": "Laptop Pro 14", "category": "laptop", "price": 1499},  
  {"id": 2, "name": "Notebook Air 13", "category": "laptop", "price": 999}  
]
```

Structured outputs are easier for both code and models to reason about than loosely formatted text.

State in an Agent

Even simple agents maintain state:

- Conversation messages
- Prior tool calls
- Current step number
- Observed tool outputs
- Final answer candidate

Without state, the system cannot reason across multiple turns or multiple actions.

```
messages: list[dict[str, Any]] = [  
    {"role": "system", "content": SYSTEM_PROMPT},  
    {"role": "user", "content": task},  
]
```

```
tool_calls: list[ToolCall] = []  
max_turns = 5
```

Error Handling in the Agent Loop

Real tools fail.
Inputs may be missing.
External APIs may timeout.
A database may return nothing.

A robust agent should:

- Catch tool errors
- Record what failed
- Avoid infinite retries
- Return a graceful response when needed

```
def execute_tool(tool_name: str, arguments: dict) -> Any:
    if tool_name not in TOOLS:
        return {"error": f"Unknown tool: {tool_name}"}
    try:
        return TOOLS[tool_name](**arguments)
    except Exception as exc:
        return {"error": f"Tool {tool_name} failed: {exc}"}
```

Retries for Transient Failures

```
def chat_completion(...):  
    response = requests.post(  
        OPENROUTER_BASE_URL,  
        headers=headers,  
        json=payload,  
        timeout=60,  
    )
```

Retry wrappers are useful for temporary failures such as network issues, but they should not hide persistent design problems.

Observability: What Happened?

When an agent behaves badly, you want to know:

- What tool did it choose?

- What arguments did it send?

- What came back?

- How many turns did it take?

- How much did it cost?

In the examples, `--verbose` gives a lightweight version of this observability without adding a larger logging stack.

Logging Agent Turns

```
for call in result.tool_calls:
    print(f"Tool: {call.tool_name}")
    print(f"Arguments:
{json.dumps(call.arguments)}")
    print(f"Result: {json.dumps(call.result)}")
```

This powers the `--verbose` mode in both the direct ReAct and MCP examples.

Tracking Cost

```
class CostTracker:
    def __init__(self):
        self.total_cost = 0.0

    def track_call(self, input_tokens: int, output_tokens: int,
in_price: float, out_price: float):
        cost = input_tokens * in_price + output_tokens *
out_price
        self.total_cost += cost
        return cost
```

Guardrails and Safety

Not every tool should be callable without restriction. The agent loop needs policy checks around tool execution.

Examples that may require extra care:

- Sending notifications
 - Writing to databases
 - Purchasing items
 - Deleting records
 - Calling expensive services
-

Simple Safety Check

```
class SafetyGuardrails:
    def __init__(self):
        self.dangerous_tools = ["send_notification"]

    def check_tool_call(self, tool_name: str, args: dict) -> bool:
        if tool_name in self.dangerous_tools:
            return False
        return True
```

Human-the-Loop Approval

```
def request_human_approval(action: str, details: dict) -> bool:
    print("APPROVAL REQUIRED")
    print(action)
    print(details)
    response = input("Approve? (yes/no): ").strip().lower()
    return response == "yes"
```

From Direct Tools to MCP

So far, the agent knows its tools because the Python program defines them directly.

That works well inside one codebase.

But what if tools should be discoverable, reusable, and shared across clients?

That is the motivation for MCP.



What MCP Standardizes



MCP = Model Context Protocol

It standardizes how a client and a tool server communicate.

Instead of every app inventing a custom tool format, MCP provides a common interface for:

- Tool discovery

- Tool schemas

- Tool invocation

- Structured results

Take a look here:

<https://modelcontextprotocol.io/docs/getting-started/intro>

MCP Architecture

LLM Agent <-> MCP Client Session <-> MCP Server <-> Tools / External APIs

Client:

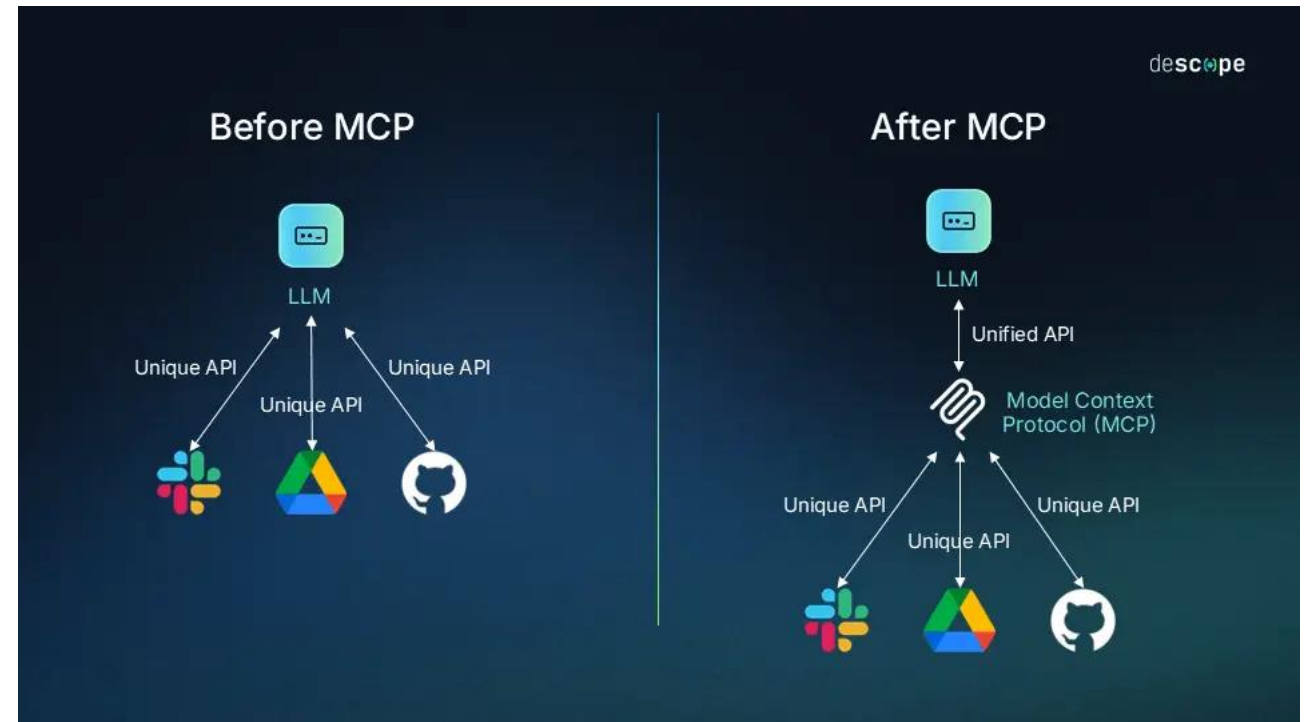
The assistant or agent that wants to use tools

MCP Server:

Exposes available tools and handles requests

Tools:

The underlying functions, APIs, or services doing the actual work



Thanks to <https://www.descope.com/learn/post/mcp> for diagram

FASTMCP Server Setup

```
from fastmcp import FastMCP
```

```
mcp = FastMCP("demo-tools")
```

A FastMCP server starts by naming a tool collection.
That server can then expose Python functions as MCP tools.

FastMCP reduces the amount of protocol boilerplate you write; the framework handles the MCP exposure layer.

Free to focus on:

- What tools exist
- What arguments they take
- What they return

Defining MCP Tools with Decorators

```
@mcp.tool
def search_database(query: str, limit: int = 10) -> list[dict]:
    normalized = query.strip().lower()
    return [
        product for product in PRODUCTS
        if normalized in product["name"].lower() or normalized in product["category"].lower()
   ][:limit]
```

```
@mcp.tool
def get_user_info(username: str) -> dict:
    return USERS[username.strip().lower()]
```

Additional MCP Tools

@mcp.tool

def get_weather(city: str) -> dict:

 latitude, longitude, resolved_city = _lookup_coordinates(city)

 current = fetch_current_weather(latitude, longitude)

 return {

 "city": resolved_city,

 "temperature_f": current["temperature_2m"],

 "conditions": _weather_code_to_description(current["weather_code"])

 }

@mcp.tool

def send_notification(username: str, message: str) -> dict:

 return {"status": "sent", "username": username, "message": message}

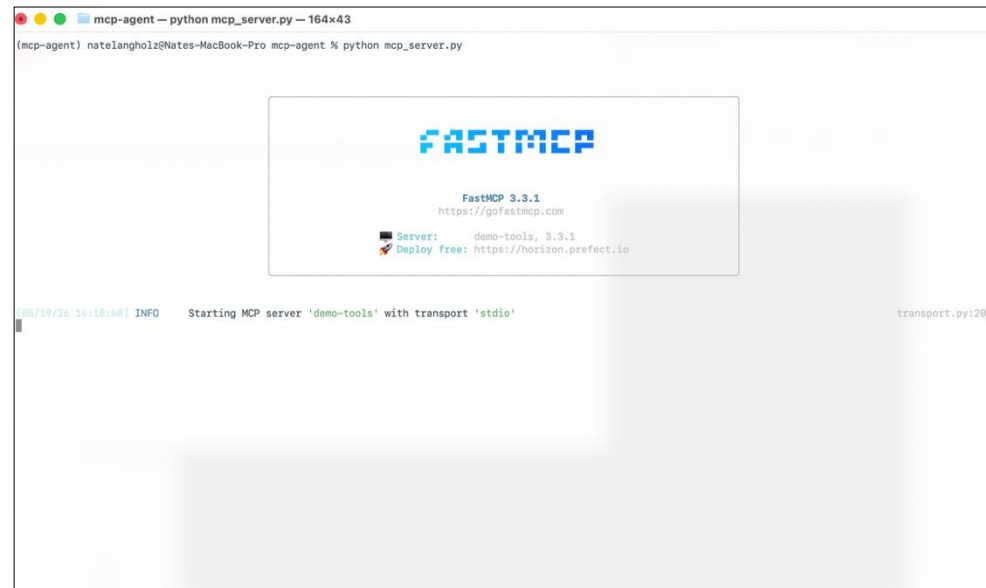
Running the MCP Server

```
if __name__ == "__main__":  
    mcp.run()
```

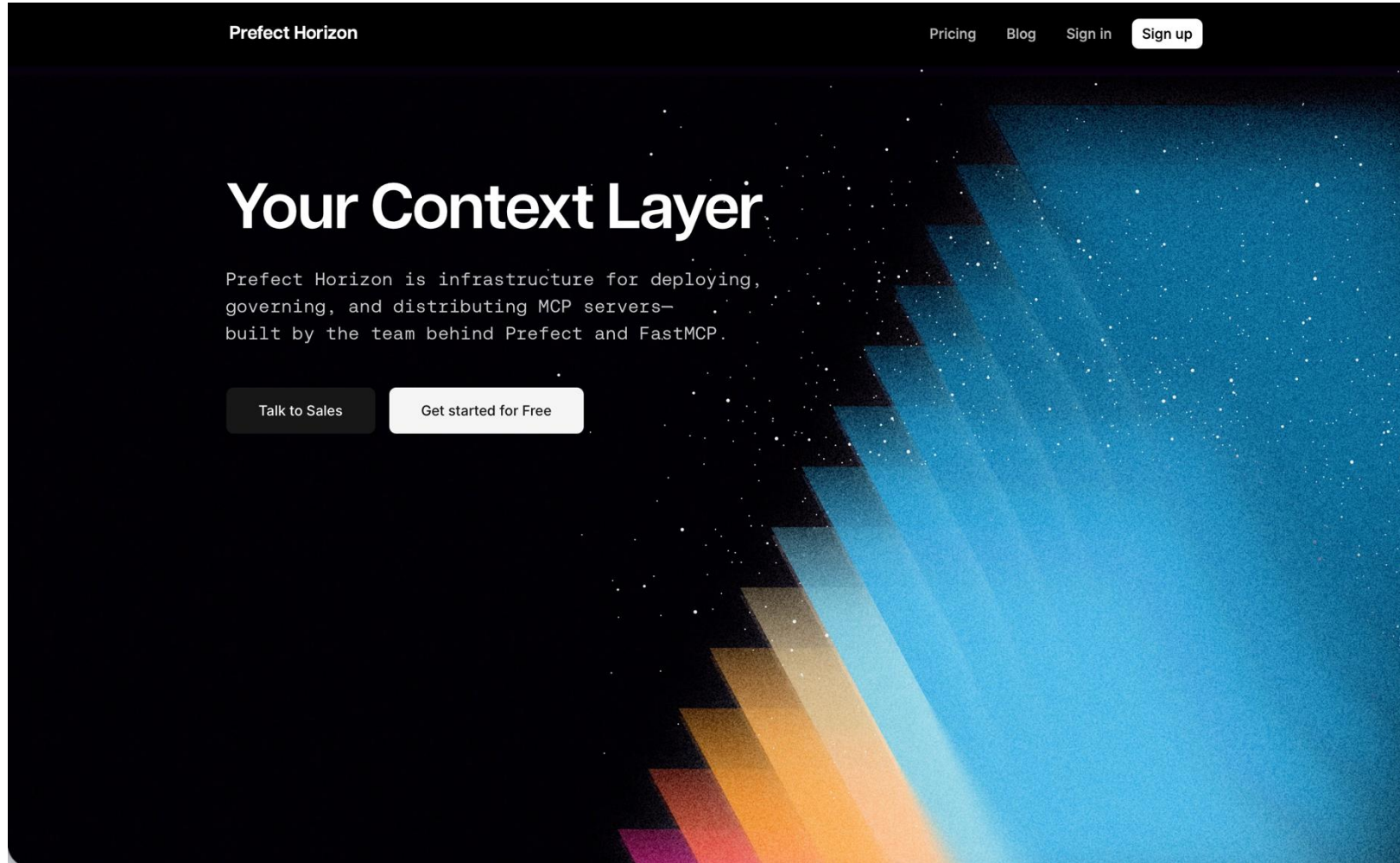
The server process stays alive and serves tool metadata and tool calls to a compatible client.

Client Configuration for MCP

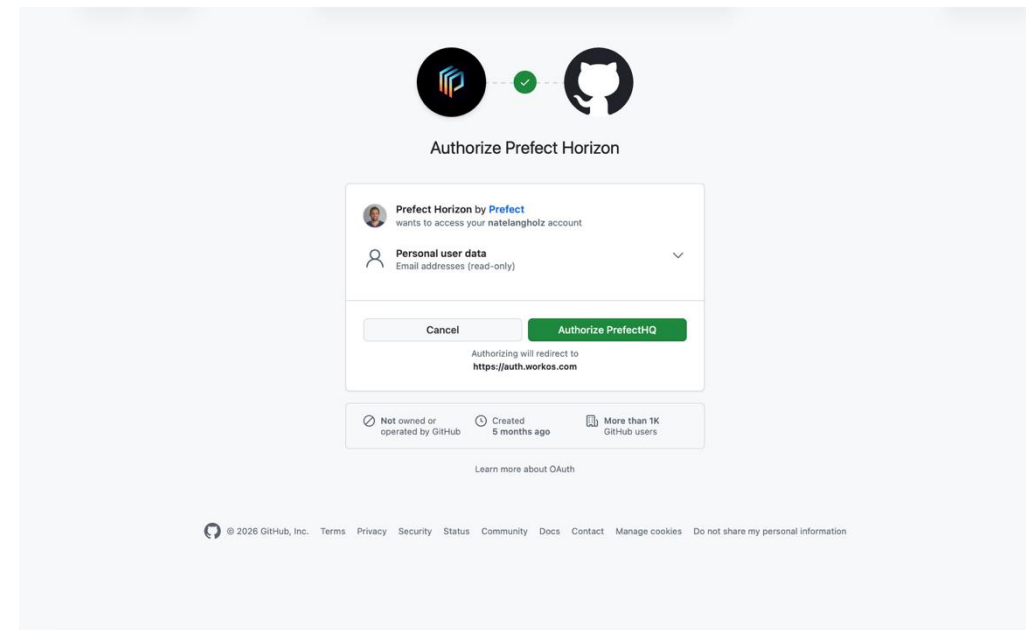
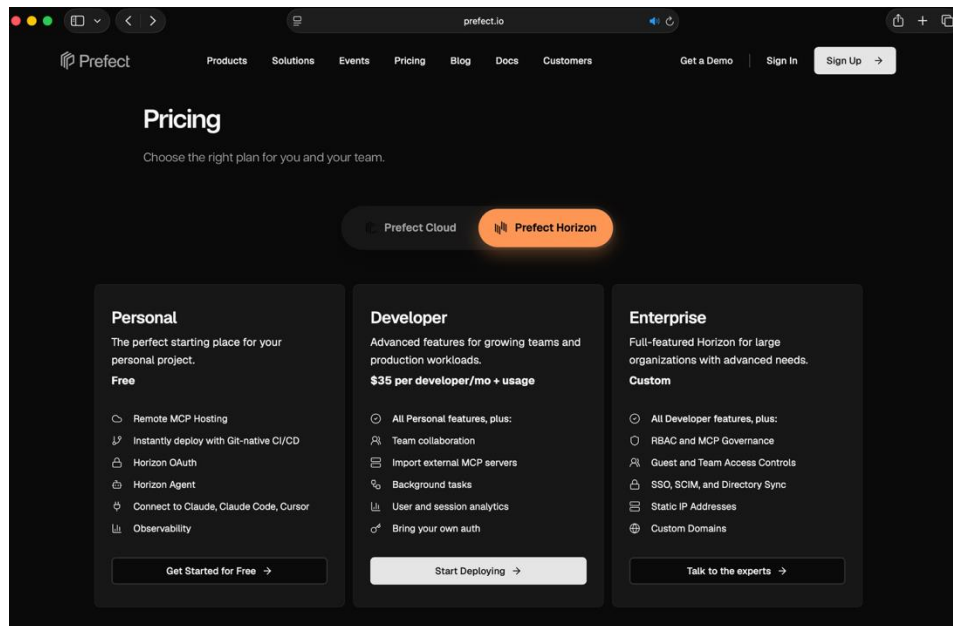
```
{  
  "mcpServers": {  
    "demo-tools": {  
      "command": "python",  
      "args": ["mcp_server.py"],  
      "cwd": "."  
    }  
  }  
}
```



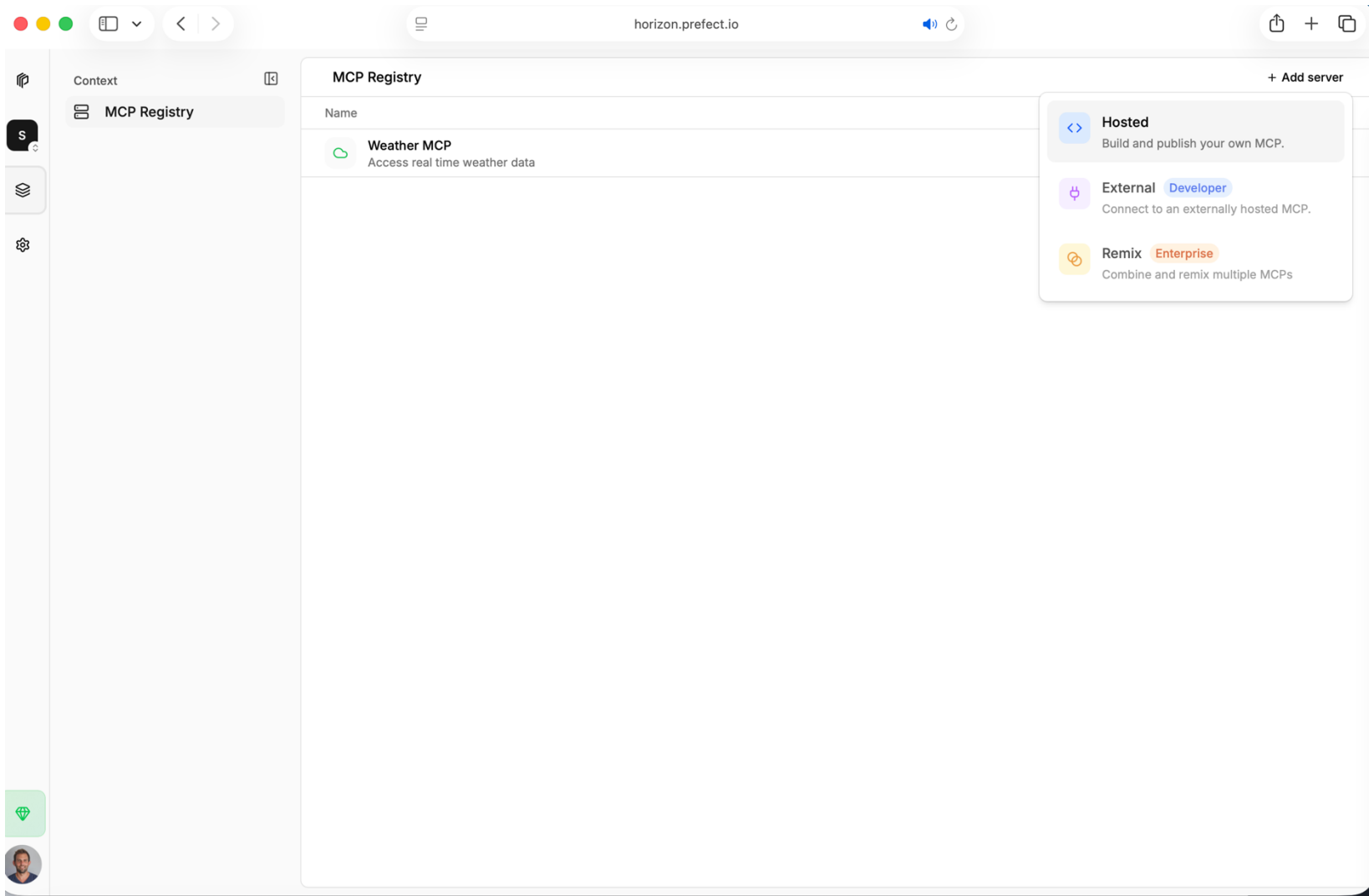
Prefect Horizon



Connect to Github to Deploy MCP Servers



MCP Registry



Direct Tools vs MCP Tools

Direct local tools:

- Agent imports Python functions directly
- Simple and fast inside one codebase
- Tight coupling between agent and tools

MCP tools:

- Agent talks to a tool server through a protocol
- Tools are more reusable and discoverable
- Looser coupling between client and tool implementation

An MCP Agent Still Follows the Same Loop

1. Receive a task
2. Initialize the MCP session
3. Discover available tools
4. Let the LLM choose a tool
5. Send arguments to the MCP server
6. Read the result
7. Continue or answer

In one-shot mode this session lasts for one request.

In interactive mode the same MCP session can stay alive across multiple user turns.

If the user clears context, later follow-ups should ask for clarification instead of guessing.

Representing MCP Tool Calls

```
@dataclass
class MCPToolCall:
    tool_name: str
    arguments: dict[str, Any]
    result: Any

@dataclass
class MCPAgentResult:
    final_answer: str
    tool_calls: list[MCPToolCall]
```

Creating a Real MCP Client Session

```
server_params = StdioServerParameters(  
    command="python",  
    args=["mcp_server.py"],  
)  
  
async with stdio_client(server_params) as (read_stream, write_stream):  
    async with ClientSession(read_stream, write_stream) as session:  
        await session.initialize()  
        # reuse this session across turns in interactive mode
```

Discovering Tools Dynamically

```
tools_result = await session.list_tools()
llm_tools = [
    {
        "type": "function",
        "function": {
            "name": tool.name,
            "description": tool.description or "",
            "parameters": tool.inputSchema,
        },
    }
    for tool in tools_result.tools
]
```

Calling MCP Tools from the Agent

```
tool_result = await session.call_tool(tool_name, arguments=arguments)
normalized_result = self._normalize_tool_result(tool_result)
```

```
messages.append({
    "role": "tool",
    "tool_call_id": tool_call["id"],
    "content": json.dumps(normalized_result),
})
```

```
# later turns can reuse the same session and prior messages
# after clear, ambiguous references should trigger clarification instead
```

Testing the MCP Agent

Testing goals:

- Verify MCP session setup works
- Verify tool schemas convert correctly for the LLM
- Verify tool results are normalized correctly
- Verify the final answer reflects multi-step tool use
- Verify that the CLI still supports default, one-shot, and interactive usage paths
- Verify that interactive mode can preserve context until the user resets it
- Verify that ambiguous follow-ups after reset trigger clarification instead of guessing

The same testing principles apply whether tools are local or protocol-based.

Direct Tooling and MCP Are Complementary

Use direct tools when:

- Everything lives in one codebase
- You want simplicity and low overhead

Use MCP when:

- Tools should be discoverable
- Multiple clients may use the same tools
- You want a standard protocol boundary

API Wrapper for an Agent

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class AgentRequest(BaseModel):
    task: str
```

```
@app.post("/agent/run")
def run_agent_endpoint(request: AgentRequest):
    result = agent.run(request.task)
    return {"answer": result.final_answer}
```

Why Expose an Agent Through an API?

Wrapping an agent in an API lets other systems call it. This turns the agent from a script into a reusable service.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN uv pip install --system -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

Run with Podman

```
podman build -t my-agent .
podman run -d -p 8000:8000 my-agent
```

At this point the agent behaves like any other containerized web service, but its internal logic includes tool orchestration.

Performance Considerations

Agent design is also a systems design problem.

Agents can become slow if they:

- Use too many turns

- Call tools serially when parallelism is possible

- Repeat identical work

- Send too much context to the model

- Use an expensive model for a simple task

Parallel Tool Calls

```
results = await asyncio.gather(  
    execute_tool_async("search_database", {"query": "laptop"}),  
    execute_tool_async("search_database", {"query": "tablet"}),  
    return_exceptions=True  
)
```

Parallel execution can reduce latency when tool calls are independent.

Caching Can Save Time and Money

If the same task or tool query appears repeatedly, caching may help.

Examples:

- Cached weather lookups

- Cached product searches

- Cached summaries for identical prompts

Caching is especially useful when the tool result is stable for a period of time.

```
def cache_key(task: str) -> str:
    return hashlib.sha256(task.encode()).hexdigest()

cached = redis_client.get(cache_key(task))
if cached:
    return json.loads(cached)

result = agent.run(task)
redis_client.setex(cache_key(task), 3600, json.dumps(result))
```

Security Concerns for Agents

Security matters more once an LLM can take actions.

Concerns include:

- Prompt injection through tool results
- Unsafe arguments passed to external systems
- Rate abuse
- Excessive spending
- Sensitive data exposure
- Improper permissions on action tools

Rate Limiting Pattern

```
class RateLimiter:
    def __init__(self, max_calls: int, window_seconds: int):
        self.max_calls = max_calls
        self.window_seconds = window_seconds
        self.calls = defaultdict(list)

    def check_limit(self, user_id: str) -> bool:
        # trim old calls, then allow or reject
        ...
```


Single-Agent vs Multi-Agent Thinking

Many tasks can be handled by one agent with good tools.

Use multiple agents only when roles are genuinely distinct.

Examples:

- Researcher agent gathers information

- Analyst agent interprets data

- Writer agent formats output

```
research = researcher.run(task)
```

```
analysis = analyst.run(research)
```

```
report = writer.run(analysis)
```

This pattern can be powerful, but it also adds latency, complexity, and more failure points. Do NOT assume multiple agents are automatically better.

Evaluation Questions

To evaluate an agent, ask:

- Did it complete the task?
- Did it use the correct tool?
- Did it stop at the right time?
- How many turns did it take?
- How much did it cost?
- How often did it fail?
- Would a user trust the output?

Common Agent Pitfalls

Poor tool descriptions

No visibility into intermediate steps

Infinite or wasteful loops

Weak error handling

Unsafe action tools

Ignoring deployment and monitoring concerns

Treating the LLM as if it were always correct

A Good Development Sequence

1. Start with direct tools and structured outputs
2. Add explicit state and logging
3. Write tests around helper logic
4. Add an LLM for tool selection
5. Add safety checks
6. Expose the agent as an API
7. Containerize and monitor it
8. Introduce MCP when tool sharing and standardization matter